

0.90.7. L'écran titre — SELECT, retour au titre

Le 0.90.5, et SELECT qui ramène au titre. Le titre se dessine alors deux fois : on le range dans une fonction, dessinerTitre().

1. Ce qu'il faut comprendre

C'est le 0.90.5, avec une chose en plus : sur l'écran d'après, **SELECT** ramène au titre. `ecranTitre` repasse à 1, et le titre se redessine.

Ce qui est nouveau ici : ranger des lignes dans une fonction (comme `tour_de_jeu` au 0.80.2). Le titre doit être dessiné **au départ**, et à **chaque retour**. Plutôt que d'écrire les mêmes lignes deux fois, on les met dans `void dessinerTitre() { ... }`, et on écrit juste `dessinerTitre();` là où il faut.

Même chose pour vider l'écran : `viderEcran()` range la boucle des 18 lignes. Elle sert pour aller sur l'écran d'après, et pour en revenir.

`void` veut dire que la fonction ne rend rien : elle **fait** quelque chose (elle dessine, elle efface), c'est tout.

Au retour, `visible` repasse à 1 : `dessinerTitre()` vient d'écrire la consigne, elle est donc à l'écran.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texteGrand>` → [0.86.3. La fonction texteGrand\(\) — un texte agrandi](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <couleurTexte>` → [0.87.7. La fonction couleurTexte\(\) — la couleur des lettres](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1710 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	63 o	← la console, à l'allumage — la mise en route de la console
texte_grand_0()	195 o	← dessinerTitre() — ta fonction
chaque_minuteur()	124 o	← main() — une fonction fournie par le langage
dessinerTitre()	23 o	← main() — ta fonction
viderEcran()	33 o	← main() — ta fonction
main()	151 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo

```

EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
LireManette       37 o ← bouton() – lit les huit boutons
Tuiles            848 o ← CopierTuiles – la police entière, et 9 tuiles dessinées
"APPUIE SUR START" 16 o ← texte() – un texte gravé
"APPUIE SUR START" 16 o ← texte() – un texte gravé
"C EST PARTI"      11 o ← texte() – un texte gravé
"SELECT : LE TIT..." 17 o ← texte() – un texte gravé
GRAND_TABLE_0_0_0  12 o ← poser() – une table gravée
GRAND_TABLE_0_1_1  12 o ← poser() – une table gravée
GRAND_TABLE_0_2_2  12 o ← poser() – une table gravée
GRAND_TABLE_0_3_3  12 o ← poser() – une table gravée
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
    poser()          x 4    122 o sur place, dans texte_grand_0() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (12 o),
GRAND_TABLE_0_1_1 (12 o), GRAND_TABLE_0_2_2 (12 o), GRAND_TABLE_0_3_3 (12 o)
    texte()          x 4    44 o sur place, dans dessinerTitre(), main() + EcrireTexte (10 o)
    effacer()         x 2    22 o sur place, dans viderEcran(), main() + AdresseCase (10 o), EffacerCases (12 o)
    couleurFond()     x 1    12 o sur place, dans main()
    bouton()          x 2    10 o sur place, dans main() + LireManette (37 o)
    image()           x 1    3 o sur place, dans main() + AttendreImage (28 o)
    images()          x 1    2 o sur place, dans chaque_minuteur()
Tes fonctions appelées :
    chaque_minuteur() x 1    10 o sur place, dans main()
    dessinerTitre()   x 2    6 o sur place, dans main()
    viderEcran()      x 2    6 o sur place, dans main()
    texte_grand_0()   x 1    3 o sur place, dans dessinerTitre()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

#include <textGrand>      // écrit un texte agrandi, taille de 0 à 10
#include <texte>          // écrit un texte à l'écran
#include <effacer>        // efface des cases, ou tout le fond
#include <couleurTexte>   // choisit la couleur des lettres
#include <chaque>         // répond 1 toutes les n millisecondes
#include <bouton>         // lit un bouton de la manette

```

2. UNE VARIABLE — ECRANTITRE

```
uint8_t ecranTitre = 1; // 1 = le titre, 0 = l'écran d'après
```

3. UNE VARIABLE — VISIBLE

```

uint8_t visible = 1; // la consigne : 1 = à l'écran, 0 = effacée

// ---- NOUVEAU : des lignes rangées sous un nom, pour s'en servir plusieurs fois ----

// Dessiner le titre : au départ, ET à chaque retour avec SELECT.

```

4. UNE FONCTION — DESSINERTITRE()

```

void dessinerTitre() {
    texteGrand(4, 5, "JEU", 3);
    texte(2, 12, "APPUIE SUR START");
}

// On vide l'écran : 18 lignes (0 à 17), 20 cases chacune.

```

5. UNE FONCTION — VIDERECRAN()

```
void viderEcran() {
    for (uint8_t y = 0; y < 18; y++) {
        effacer(0, y, 20);
    }
}
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    couleurTexte(31, 16, 0);
    dessinerTitre();           // la première fois : toutes les lignes de la fonction

    while (true) {
        image();

        // La consigne qui clignote (le 0.90.5).
        if (ecranTitre == 1 && chaque(500)) {
            if (visible == 1) {
                effacer(2, 12, "APPUIE SUR START");
            } else {
                texte(2, 12, "APPUIE SUR START");
            }
            visible = 1 - visible;
        }

        // Titre → écran d'après, avec START.
        if (ecranTitre == 1 && bouton(START)) {
            ecranTitre = 0;
            viderEcran();       // la boucle des 18 lignes, rangée dans sa fonction
            texte(5, 8, "C EST PARTI");
            texte(1, 16, "SELECT : LE TITRE");
        }

        // ---- NOUVEAU : écran d'après → titre, avec SELECT.
        if (ecranTitre == 0 && bouton(SELECT)) {
            ecranTitre = 1;     // on est de nouveau sur le titre
            viderEcran();
            dessinerTitre();    // la deuxième fois : les mêmes lignes, sans les réécrire
            visible = 1;       // la consigne vient d'être écrite : elle est là
        }
    }
}
```

3. Ce qui se passe, étape par étape

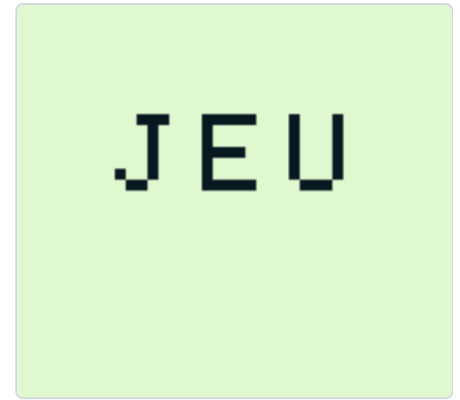
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



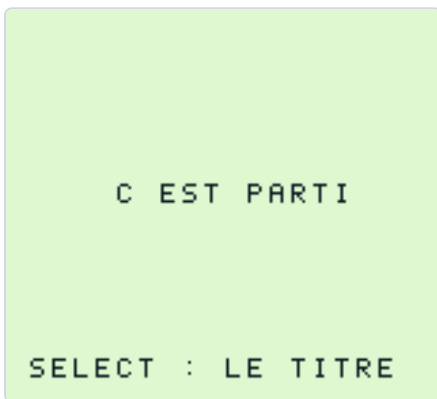
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



START — l'écran a changé

5. Ce qu'on doit voir

Le titre ; START : C EST PARTI ; SELECT : le titre revient, et la consigne clignote de nouveau.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ START : l'écran d'après
- ✓ SELECT : retour au titre (`ecranTitre = 1`)
- ✓ le grand JEU est redessiné, C EST PARTI a disparu

Ensuite : **0.90.8. L'écran titre — trois écrans** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++